ALLIANCE

UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE MANAGEMENT SYSTEM

CAB BOOKING MANAGEMENT SYSTEM

ASSIGNMENT 6

IMPLEMENTATION OF DATA INTEGRITY CONSTRAINTS

Submitted by: ***Preksha V***
Register Number: ***2511021240063***
Programme: Bachelors of Computer Application - III

Submitted to: ***Renita Joy R***
Subject: Database Management System (DBMS)

1. Introduction

Data integrity means that the data stored in the database is accurate, consistent and follows the rules of the business. In a relational database, integrity is enforced by constraints. This assignment applies primary keys, foreign keys, UNIQUE, CHECK, DEFAULT and NOT NULL constraints to the eight tables of the Cab Booking Management System, explains why each one is needed, and then runs SQL statements to prove that invalid data is rejected and that valid data keeps the tables correctly connected.

Most constraints were defined in the CREATE TABLE statements of Assignment 5. In this assignment they are reviewed and explained, a few extra constraints are added with ALTER TABLE, and every constraint is tested.

2. Primary Keys

A primary key uniquely identifies each row and can never be NULL. Every table has a single-column primary key, which also satisfies 2NF (no partial dependency), as analysed in Assignment 3.

Table	Primary key	Purpose
CUSTOMER	Customer_ID	Identifies one customer
DRIVER	Driver_ID	Identifies one driver
CAB	Cab_ID	Identifies one vehicle
BOOKING	Booking_ID	Identifies one reservation
TRIP	Trip_ID	Identifies one journey
PAYMENT	Payment_ID	Identifies one payment
RATING	Rating_ID	Identifies one feedback record
DRIVER_LOCATION	Location_ID	Identifies one location update

3. Foreign Keys

A foreign key is a column that must contain a value that already exists in the primary key of another table. It connects the tables and prevents orphan records. By default a parent row cannot be deleted or changed while child rows still refer to it (RESTRICT).

Foreign key	References	Relationship	Delete rule
CAB.Driver_ID	DRIVER(Driver_ID)	Driver - Cab (1:1, with UNIQUE)	RESTRICT
BOOKING.Customer_ID	CUSTOMER(Customer_ID)	Customer - Booking (1:M)	RESTRICT
BOOKING.Driver_ID	DRIVER(Driver_ID)	Driver - Booking (1:M)	RESTRICT
BOOKING.Cab_ID	CAB(Cab_ID)	Cab - Booking (1:M)	RESTRICT
TRIP.Booking_ID	BOOKING(Booking_ID)	Booking - Trip (1:1, with UNIQUE)	RESTRICT
PAYMENT.Booking_ID	BOOKING(Booking_ID)	Booking - Payment (1:1, with UNIQUE)	RESTRICT
RATING.Booking_ID	BOOKING(Booking_ID)	Booking - Rating (1:0..1, with UNIQUE)	RESTRICT
DRIVER_LOCATION.Driver_ID	DRIVER(Driver_ID)	Driver - Driver_Location (1:M)	CASCADE (see section 5)

4. UNIQUE, CHECK, DEFAULT and NOT NULL Constraints

4.1 UNIQUE constraints

Column	Table	Purpose
Phone, Email	CUSTOMER	One account per phone number and email address
Phone, License_Number	DRIVER	A driver and a driving licence are registered only once
Vehicle_Number	CAB	One registration number belongs to only one cab
Driver_ID	CAB	Enforces the Driver-Cab 1:1 rule
Booking_ID	TRIP	Enforces Booking-Trip 1:1
Booking_ID	PAYMENT	Enforces Booking-Payment 1:1
Booking_ID	RATING	Enforces Booking-Rating 1:0..1 (at most one rating)

4.2 CHECK constraints

Rule	Table	Purpose
Status IN ('Available','Busy','Offline')	DRIVER	Only valid driver states
Cab_Type IN ('Sedan','SUV','Hatchback')	CAB	Only valid vehicle types
Status IN ('Available','Booked','Maintenance')	CAB	Only valid vehicle states
Booking_Status IN ('Pending','Confirmed','Cancelled','Completed')	BOOKING	Only valid booking states
Distance >= 0 and Fare >= 0	TRIP	No negative distance or fare
End_Time IS NULL OR End_Time >= Start_Time	TRIP	A trip cannot end before it starts
Trip_Status IN ('Started','Completed','Cancelled')	TRIP	Only valid trip states
Amount >= 0	PAYMENT	No negative payment
Payment_Method IN ('UPI','Card','Cash')	PAYMENT	Only supported payment methods
Payment_Status IN ('Successful','Pending','Failed')	PAYMENT	Only valid payment states
Rating BETWEEN 1 AND 5	RATING	Ratings stay on the 1 to 5 scale
CHAR_LENGTH(Phone) >= 10 (added in section 5)	CUSTOMER, DRIVER	Rejects incomplete phone numbers
Pickup_Location <> Drop_Location (added in section 5)	BOOKING	A booking must go to a different place

4.3 DEFAULT constraints

Column	Default value	Purpose
DRIVER.Status	Available	A newly registered driver can accept bookings
CAB.Status	Available	A new cab is ready for use
BOOKING.Booking_Status	Pending	A new booking waits for confirmation
TRIP.Trip_Status	Started	A trip record is created when the journey begins
PAYMENT.Payment_Status	Pending	A payment is pending until it is completed

4.4 NOT NULL constraints

NOT NULL is used for columns that must always have a value: names, phone numbers, licence and vehicle numbers, booking date and locations, all foreign keys of BOOKING, TRIP, PAYMENT, RATING and DRIVER_LOCATION, the payment amount and method, the rating value, and the coordinates. Optional columns such as Email, Address, Review, End_Time and Payment_Date allow NULL.

5. Adding Constraints with ALTER TABLE

Constraints can also be added after a table has been created. Three extra CHECK constraints are added, and the foreign key of DRIVER_LOCATION is replaced by one with ON DELETE CASCADE and ON UPDATE CASCADE. Location updates are only tracking data, so they can be deleted together with the driver, whereas bookings, trips and payments must be kept as history and therefore use RESTRICT.

```
ALTER TABLE CUSTOMER ADD CONSTRAINT chk_customer_phone CHECK (CHAR_LENGTH(Phone) >= 10);
ALTER TABLE DRIVER ADD CONSTRAINT chk_driver_phone CHECK (CHAR_LENGTH(Phone) >= 10);
ALTER TABLE BOOKING ADD CONSTRAINT chk_booking_locations CHECK (Pickup_Location <> Drop_Location);
ALTER TABLE DRIVER_LOCATION DROP FOREIGN KEY DRIVER_LOCATION_ibfk_1;
ALTER TABLE DRIVER_LOCATION ADD CONSTRAINT fk_location_driver FOREIGN KEY (Driver_ID) REFERENCES
    DRIVER(Driver_ID) ON DELETE CASCADE ON UPDATE CASCADE;
```

6. Verification of Data Integrity

The statements below were executed on the database. A small set of valid sample rows is inserted first. Then invalid statements are attempted: each one should be rejected by the matching constraint. Finally, queries are used to confirm that the relationships between the tables are intact.

Note: the statements were run on a MySQL-compatible server (MariaDB 10.11). The error numbers for primary key, foreign key and UNIQUE violations are the same in MySQL 8. For CHECK violations MySQL 8 shows an error of the form "Check constraint '...' is violated" instead of the message shown here.

6.1 Baseline valid data

One valid record is inserted into every table, starting with the parent tables.

```
mysql> INSERT INTO CUSTOMER VALUES (101, 'Rahul Sharma', '9886012101', 'rahul.sharma@gmail.com',
    -> 'Anekal, Bengaluru');
Query OK
mysql> INSERT INTO DRIVER VALUES (201, 'Ramesh Kumar', '9845012301', 'KA0120190012345', 'Available');
Query OK
mysql> INSERT INTO CAB VALUES (301, 201, 'KA01AB1234', 'Sedan', 'Maruti Swift Dzire', 'Available');
Query OK
mysql> INSERT INTO BOOKING VALUES (5001, 101, 201, 301, '2026-10-01 08:15:00', 'Alliance University',
    -> 'Electronic City Phase 1', 'Completed');
Query OK
mysql> INSERT INTO TRIP VALUES (7001, 5001, '2026-10-01 08:25:00', '2026-10-01 09:02:00', 18.50,
    -> 350.00, 'Completed');
Query OK
mysql> INSERT INTO PAYMENT VALUES (8001, 5001, 350.00, 'UPI', '2026-10-01 09:05:00', 'Successful');
Query OK
mysql> INSERT INTO RATING VALUES (9001, 5001, 5, 'Good ride');
Query OK
mysql> INSERT INTO DRIVER_LOCATION VALUES (1, 201, 12.916500, 77.610100, '2026-10-24 09:00:00');
Query OK
```

6.2 Primary key constraint

Test 1: Duplicate primary key

A second customer with Customer_ID 101 must be rejected.

```
mysql> INSERT INTO CUSTOMER VALUES (101, 'Another Person', '9000000001', 'a@x.com', 'Bengaluru');
ERROR 1062 (23000): Duplicate entry '101' for key 'PRIMARY'
```

Test 2: NULL primary key

A primary key column cannot be left empty.

```
mysql> INSERT INTO CUSTOMER VALUES (NULL, 'No Id', '9000000002', 'b@x.com', 'Bengaluru');
ERROR 1048 (23000): Column 'Customer_ID' cannot be null
```

6.3 Foreign key constraints

Test 3: Booking for a customer that does not exist

BOOKING.Customer_ID must match an existing CUSTOMER row.

```
mysql> INSERT INTO BOOKING VALUES (5002, 999, 201, 301, '2026-10-02 09:30:00', 'HSR Layout', 'MG Road',
    -> 'Pending');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
      (`CabBookingDB`.`BOOKING`, CONSTRAINT `BOOKING_ibfk_1` FOREIGN KEY (`Customer_ID`) REFERENCES
      `CUSTOMER` (`Customer_ID`))
```

Test 4: Booking with a cab that does not exist

BOOKING.Cab_ID must match an existing CAB row.

```
mysql> INSERT INTO BOOKING VALUES (5002, 101, 201, 999, '2026-10-02 09:30:00', 'HSR Layout', 'MG Road',
    -> 'Pending');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
      (`CabBookingDB`.`BOOKING`, CONSTRAINT `BOOKING_ibfk_3` FOREIGN KEY (`Cab_ID`) REFERENCES `CAB`
      (`Cab_ID`))
```

Test 5: Payment for a booking that does not exist

PAYMENT.Booking_ID must match an existing BOOKING row.

```
mysql> INSERT INTO PAYMENT VALUES (8002, 6000, 200.00, 'UPI', NULL, 'Pending');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
      (`CabBookingDB`.`PAYMENT`, CONSTRAINT `PAYMENT_ibfk_1` FOREIGN KEY (`Booking_ID`) REFERENCES
      `BOOKING` (`Booking_ID`))
```

Test 6: Deleting a parent row that has child rows

A customer who already has bookings cannot be deleted (default RESTRICT rule), so booking history is protected.

```
mysql> DELETE FROM CUSTOMER WHERE Customer_ID = 101;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
      (`CabBookingDB`.`BOOKING`, CONSTRAINT `BOOKING_ibfk_1` FOREIGN KEY (`Customer_ID`) REFERENCES
      `CUSTOMER` (`Customer_ID`))
```

Test 7: Deleting a driver who has bookings

Driver 201 is referenced by CAB, BOOKING and DRIVER_LOCATION; deletion is blocked because of CAB and BOOKING.

```
mysql> DELETE FROM DRIVER WHERE Driver_ID = 201;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
      (`CabBookingDB`.`BOOKING`, CONSTRAINT `BOOKING_ibfk_2` FOREIGN KEY (`Driver_ID`) REFERENCES
      `DRIVER` (`Driver_ID`))
```

6.4 UNIQUE constraints

Test 8: Duplicate customer phone number

No two customers can share the same phone number.

```
mysql> INSERT INTO CUSTOMER VALUES (102, 'Ananya Iyer', '9886012101', 'ananya@x.com', 'HSR Layout');
ERROR 1062 (23000): Duplicate entry '9886012101' for key 'Phone'
```

Test 9: Duplicate driver licence number

A licence number can belong to only one driver.

```
mysql> INSERT INTO DRIVER VALUES (202, 'Suresh Babu', '9845012302', 'KA0120190012345', 'Available');
ERROR 1062 (23000): Duplicate entry 'KA0120190012345' for key 'License_Number'
```

Test 10: Duplicate vehicle registration number

A vehicle number can be registered only once.

```
mysql> INSERT INTO DRIVER VALUES (202, 'Suresh Babu', '9845012302', 'KA0520180023456', 'Available');
Query OK

mysql> INSERT INTO CAB VALUES (302, 202, 'KA01AB1234', 'SUV', 'Toyota Innova Crysta', 'Available');
ERROR 1062 (23000): Duplicate entry 'KA01AB1234' for key 'Vehicle_Number'
```

Test 11: Second cab for the same driver (Driver-Cab 1:1)

UNIQUE on CAB.Driver_ID enforces the one-driver-one-cab rule.

```
mysql> INSERT INTO CAB VALUES (303, 201, 'KA03CD7788', 'Hatchback', 'Maruti WagonR', 'Available');
ERROR 1062 (23000): Duplicate entry '201' for key 'Driver_ID'
```

Test 12: Second trip for the same booking (Booking-Trip 1:1)

UNIQUE on TRIP.Booking_ID allows only one trip per booking.

```
mysql> INSERT INTO TRIP VALUES (7002, 5001, '2026-10-01 10:00:00', '2026-10-01 10:30:00', 5.00, 100.00,
    -> 'Completed');
ERROR 1062 (23000): Duplicate entry '5001' for key 'Booking_ID'
```

Test 13: Second payment for the same booking (Booking-Payment 1:1)

UNIQUE on PAYMENT.Booking_ID allows only one payment per booking.

```
mysql> INSERT INTO PAYMENT VALUES (8002, 5001, 350.00, 'Cash', NULL, 'Pending');
ERROR 1062 (23000): Duplicate entry '5001' for key 'Booking_ID'
```

Test 14: Second rating for the same booking (Booking-Rating 1:0..1)

UNIQUE on RATING.Booking_ID allows at most one rating per booking.

```
mysql> INSERT INTO RATING VALUES (9002, 5001, 4, 'Second rating');
ERROR 1062 (23000): Duplicate entry '5001' for key 'Booking_ID'
```

6.5 CHECK constraints

Test 15: Rating outside 1 to 5

Rating must be between 1 and 5.

```
mysql> INSERT INTO BOOKING VALUES (5002, 101, 201, 301, '2026-10-02 09:30:00', 'HSR Layout', 'MG Road',
    -> 'Completed');
Query OK

mysql> INSERT INTO RATING VALUES (9002, 5002, 6, 'Too high');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_1` failed for `CabBookingDB`.`RATING`
```

Test 16: Negative fare

Distance and Fare cannot be negative.

```
mysql> INSERT INTO TRIP VALUES (7002, 5002, '2026-10-02 09:40:00', '2026-10-02 10:10:00', 10.00,
    -> -50.00, 'Completed');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_2` failed for `CabBookingDB`.`TRIP`
```

Test 17: Trip ending before it starts

End_Time cannot be earlier than Start_Time.

```
mysql> INSERT INTO TRIP VALUES (7002, 5002, '2026-10-02 10:10:00', '2026-10-02 09:40:00', 10.00,
    -> 200.00, 'Completed');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_3` failed for `CabBookingDB`.`TRIP`
```

Test 18: Invalid cab type

Cab_Type is restricted to Sedan, SUV or Hatchback.

```
mysql> INSERT INTO DRIVER VALUES (203, 'Manjunath Gowda', '9845012303', 'KA0320200034567',
    -> 'Available');
Query OK

mysql> INSERT INTO CAB VALUES (303, 203, 'KA03CD7788', 'Truck', 'Tata Ace', 'Available');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_1` failed for `CabBookingDB`.`CAB`
```

Test 19: Invalid payment method

Payment_Method is restricted to UPI, Card or Cash.

```
mysql> INSERT INTO PAYMENT VALUES (8002, 5002, 200.00, 'Bitcoin', NULL, 'Pending');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_2` failed for `CabBookingDB`.`PAYMENT`
```

Test 20: Invalid driver status

Driver Status is restricted to Available, Busy or Offline.

```
mysql> INSERT INTO DRIVER VALUES (204, 'Imran Khan', '9845012304', 'KA0220170045678', 'Sleeping');
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_1` failed for `CabBookingDB`.`DRIVER`
```

Test 21: Pickup and drop location are the same

chk_booking_locations: a booking must go somewhere different.

```
mysql> INSERT INTO BOOKING VALUES (5003, 101, 201, 301, '2026-10-03 09:00:00', 'MG Road', 'MG Road',
    -> 'Pending');
ERROR 4025 (23000): CONSTRAINT `chk_booking_locations` failed for `CabBookingDB`.`BOOKING`
```

Test 22: Phone number shorter than 10 digits

chk_customer_phone: phone numbers must have at least 10 characters.

```
mysql> INSERT INTO CUSTOMER VALUES (103, 'Karthik Nair', '98860', 'karthik@x.com', 'Koramangala');
ERROR 4025 (23000): CONSTRAINT `chk_customer_phone` failed for `CabBookingDB`.`CUSTOMER`
```

6.6 NOT NULL constraint

Test 23: Customer without a name

Name is mandatory for every customer.

```
mysql> INSERT INTO CUSTOMER (Customer_ID, Phone) VALUES (103, '9886012103');
ERROR 1364 (HY000): Field 'Name' doesn't have a default value
```

6.7 DEFAULT constraints

Test 24: Rows inserted without status values

Status columns take their default value when no value is supplied (Driver = Available, Booking = Pending, Trip = Started, Payment = Pending).

```
mysql> INSERT INTO DRIVER (Driver_ID, Name, Phone, License_Number) VALUES (205, 'Venkatesh Rao',
    -> '9845012305', 'KA0420190056789');
Query OK

mysql> INSERT INTO BOOKING (Booking_ID, Customer_ID, Driver_ID, Cab_ID, Booking_Date, Pickup_Location,
    -> Drop_Location) VALUES (5004, 101, 201, 301, '2026-10-04 08:00:00', 'Anekal', 'Majestic');
Query OK

mysql> INSERT INTO TRIP (Trip_ID, Booking_ID) VALUES (7004, 5004);
Query OK

mysql> INSERT INTO PAYMENT (Payment_ID, Booking_ID, Amount, Payment_Method) VALUES (8004, 5004, 150.00,
    -> 'Cash');
Query OK

mysql> SELECT Driver_ID, Status FROM DRIVER WHERE Driver_ID = 205;
+-----------+-----------+
| Driver_ID | Status    |
+-----------+-----------+
|       205 | Available |
+-----------+-----------+

mysql> SELECT Booking_ID, Booking_Status FROM BOOKING WHERE Booking_ID = 5004;
+------------+----------------+
| Booking_ID | Booking_Status |
+------------+----------------+
|       5004 | Pending        |
+------------+----------------+

mysql> SELECT Trip_ID, Trip_Status FROM TRIP WHERE Trip_ID = 7004;
+---------+-------------+
| Trip_ID | Trip_Status |
+---------+-------------+
|    7004 | Started     |
+---------+-------------+

mysql> SELECT Payment_ID, Payment_Status FROM PAYMENT WHERE Payment_ID = 8004;
+------------+----------------+
| Payment_ID | Payment_Status |
+------------+----------------+
|       8004 | Pending        |
+------------+----------------+
```

6.8 Referential action (ON DELETE CASCADE)

Test 25: Deleting a driver removes only that driver's location pings

DRIVER_LOCATION was re-created with ON DELETE CASCADE because location updates are not history that must be kept. A driver with no bookings or cab can be deleted and the location rows disappear automatically.

```
mysql> INSERT INTO DRIVER VALUES (299, 'Test Driver', '9000000299', 'KA9999999999999', 'Offline');
Query OK

mysql> INSERT INTO DRIVER_LOCATION VALUES (90, 299, 12.9000, 77.6000, '2026-10-24 10:00:00');
Query OK

mysql> INSERT INTO DRIVER_LOCATION VALUES (91, 299, 12.9100, 77.6100, '2026-10-24 10:05:00');
Query OK

mysql> SELECT Location_ID, Driver_ID FROM DRIVER_LOCATION WHERE Driver_ID = 299;
+-------------+-----------+
| Location_ID | Driver_ID |
+-------------+-----------+
|          90 |       299 |
|          91 |       299 |
+-------------+-----------+

mysql> DELETE FROM DRIVER WHERE Driver_ID = 299;
Query OK

mysql> SELECT COUNT(*) AS Remaining_Locations FROM DRIVER_LOCATION WHERE Driver_ID = 299;
+---------------------+
| Remaining_Locations |
+---------------------+
|                   0 |
+---------------------+
```

6.9 Checking that relationships are maintained

Test 26: Joining all tables through the foreign keys

One booking is retrieved together with its customer, driver, cab, trip, payment and rating.

```
mysql> SELECT b.Booking_ID, c.Name AS Customer, d.Name AS Driver, cb.Vehicle_Number, t.Distance,
    -> t.Fare, p.Payment_Method, r.Rating FROM BOOKING b JOIN CUSTOMER c ON b.Customer_ID =
    -> c.Customer_ID JOIN DRIVER d ON b.Driver_ID = d.Driver_ID JOIN CAB cb ON b.Cab_ID = cb.Cab_ID
    -> LEFT JOIN TRIP t ON t.Booking_ID = b.Booking_ID LEFT JOIN PAYMENT p ON p.Booking_ID =
    -> b.Booking_ID LEFT JOIN RATING r ON r.Booking_ID = b.Booking_ID WHERE b.Booking_ID = 5001;
+------------+--------------+--------------+----------------+----------+--------+----------------+--
      ------+
| Booking_ID | Customer     | Driver       | Vehicle_Number | Distance | Fare   | Payment_Method |
      Rating |
+------------+--------------+--------------+----------------+----------+--------+----------------+--
      ------+
|       5001 | Rahul Sharma | Ramesh Kumar | KA01AB1234     |    18.50 | 350.00 | UPI            |
      5 |
+------------+--------------+--------------+----------------+----------+--------+----------------+--
      ------+
```

Test 27: Orphan record check

Each query looks for child rows without a parent row. Every count must be 0.

```
mysql> SELECT COUNT(*) AS Orphan_Bookings FROM BOOKING b LEFT JOIN CUSTOMER c ON b.Customer_ID =
    -> c.Customer_ID WHERE c.Customer_ID IS NULL;
+-----------------+
| Orphan_Bookings |
+-----------------+
|               0 |
+-----------------+

mysql> SELECT COUNT(*) AS Orphan_Trips FROM TRIP t LEFT JOIN BOOKING b ON t.Booking_ID = b.Booking_ID
    -> WHERE b.Booking_ID IS NULL;
+--------------+
| Orphan_Trips |
+--------------+
|            0 |
+--------------+

mysql> SELECT COUNT(*) AS Orphan_Payments FROM PAYMENT p LEFT JOIN BOOKING b ON p.Booking_ID =
    -> b.Booking_ID WHERE b.Booking_ID IS NULL;
+-----------------+
| Orphan_Payments |
+-----------------+
|               0 |
+-----------------+

mysql> SELECT COUNT(*) AS Orphan_Ratings FROM RATING r LEFT JOIN BOOKING b ON r.Booking_ID =
    -> b.Booking_ID WHERE b.Booking_ID IS NULL;
+----------------+
| Orphan_Ratings |
+----------------+
|              0 |
+----------------+
```

Test 28: List of all foreign keys defined in the database

The data dictionary confirms every foreign key and the table it references.

```
mysql> SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME FROM
    -> information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA = 'CabBookingDB' AND
    -> REFERENCED_TABLE_NAME IS NOT NULL ORDER BY TABLE_NAME, COLUMN_NAME;
+-----------------+-------------+----------------------+------------------------+
| TABLE_NAME      | COLUMN_NAME | REFERENCED_TABLE_NAME | REFERENCED_COLUMN_NAME |
+-----------------+-------------+----------------------+------------------------+
| BOOKING         | Cab_ID      | CAB                  | Cab_ID                 |
| BOOKING         | Customer_ID | CUSTOMER             | Customer_ID            |
| BOOKING         | Driver_ID   | DRIVER               | Driver_ID              |
| CAB             | Driver_ID   | DRIVER               | Driver_ID              |
| DRIVER_LOCATION | Driver_ID   | DRIVER               | Driver_ID              |
| PAYMENT         | Booking_ID  | BOOKING              | Booking_ID             |
| RATING          | Booking_ID  | BOOKING              | Booking_ID             |
| TRIP            | Booking_ID  | BOOKING              | Booking_ID             |
+-----------------+-------------+----------------------+------------------------+
```

7. Summary of Test Results

Test	Constraint tested	Result
1. Duplicate primary key	Primary key constraint	Rejected (error 1062)
2. NULL primary key	Primary key constraint	Rejected (error 1048)
3. Booking for a customer that does not exist	Foreign key constraints	Rejected (error 1452)
4. Booking with a cab that does not exist	Foreign key constraints	Rejected (error 1452)
5. Payment for a booking that does not exist	Foreign key constraints	Rejected (error 1452)
6. Deleting a parent row that has child rows	Foreign key constraints	Rejected (error 1451)
7. Deleting a driver who has bookings	Foreign key constraints	Rejected (error 1451)
8. Duplicate customer phone number	UNIQUE constraints	Rejected (error 1062)
9. Duplicate driver licence number	UNIQUE constraints	Rejected (error 1062)
10. Duplicate vehicle registration number	UNIQUE constraints	Rejected (error 1062)
11. Second cab for the same driver (Driver-Cab 1:1)	UNIQUE constraints	Rejected (error 1062)
12. Second trip for the same booking (Booking-Trip 1:1)	UNIQUE constraints	Rejected (error 1062)
13. Second payment for the same booking (Booking-Payment 1:1)	UNIQUE constraints	Rejected (error 1062)
14. Second rating for the same booking (Booking-Rating 1:0..1)	UNIQUE constraints	Rejected (error 1062)
15. Rating outside 1 to 5	CHECK constraints	Rejected (error 4025)
16. Negative fare	CHECK constraints	Rejected (error 4025)
17. Trip ending before it starts	CHECK constraints	Rejected (error 4025)
18. Invalid cab type	CHECK constraints	Rejected (error 4025)
19. Invalid payment method	CHECK constraints	Rejected (error 4025)
20. Invalid driver status	CHECK constraints	Rejected (error 4025)
21. Pickup and drop location are the same	CHECK constraints	Rejected (error 4025)
22. Phone number shorter than 10 digits	CHECK constraints	Rejected (error 4025)
23. Customer without a name	NOT NULL constraint	Rejected (error 1364)
24. Rows inserted without status values	DEFAULT constraints	Accepted / verified
25. Deleting a driver removes only that driver's location pings	Referential action (ON DELETE CASCADE)	Accepted / verified
26. Joining all tables through the foreign keys	Checking that relationships are maintained	Accepted / verified
27. Orphan record check	Checking that relationships are maintained	Accepted / verified
28. List of all foreign keys defined in the database	Checking that relationships are maintained	Accepted / verified

8. Cleaning Up the Test Data

The baseline test rows are deleted, starting with the child tables, so that the tables are empty before the sample data of Assignment 7 is inserted. The order itself also demonstrates the foreign keys: a parent row can only be deleted after its child rows are gone.

```
mysql> DELETE FROM RATING;
Query OK
mysql> DELETE FROM PAYMENT;
Query OK
mysql> DELETE FROM TRIP;
Query OK
mysql> DELETE FROM BOOKING;
Query OK
mysql> DELETE FROM DRIVER_LOCATION;
Query OK
mysql> DELETE FROM CAB;
Query OK
mysql> DELETE FROM DRIVER;
Query OK
mysql> DELETE FROM CUSTOMER;
Query OK
```

9. Conclusion

Primary keys, foreign keys, UNIQUE, CHECK, DEFAULT and NOT NULL constraints were applied to all eight tables and explained. The tests showed that duplicate keys, missing parent records, duplicate unique values, invalid statuses, out-of-range ratings and negative amounts are all rejected, that default values are filled in automatically, and that the one-to-one and one-to-many relationships are enforced by the database itself. The orphan-record queries returned zero violations, so the data remains consistent.